

Comprehensive Analysis of VSFSck: Very Simple File System Checker

I. File System Consistency Checking: Theoretical Foundation

File system consistency checking is a critical maintenance operation in any storage system. When a file system is not properly unmounted (due to power loss, system crash, etc.), its metadata can become inconsistent with its actual data structures. VSFSck is designed to detect and repair these inconsistencies in the Very Simple File System (VSFS).

The fundamental principles of file system checking include:

1. **Structural Validation:** Ensuring metadata structures conform to expected formats
2. **Reference Validation:** Verifying that all references to blocks are valid
3. **Allocation Consistency:** Confirming that allocation bitmaps match actual resource usage
4. **Duplicate Detection:** Identifying blocks claimed by multiple files
5. **Orphan Detection:** Finding unreferenced but allocated resources

Let's proceed with a detailed analysis of each component of our implementation.

II. Headers and Constants: Building the Foundation

```
#include <fcntl.h>    // File control options
#include <stdio.h>    // Standard I/O functions
#include <stdlib.h>    // Standard library functions
#include <stdint.h>    // Fixed-width integer types
#include <string.h>    // String manipulation functions
#include <unistd.h>    // POSIX API functions
#include <time.h>      // Time-related functions
```

These header files provide essential functionality:

- `fcntl.h` : Provides file control operations, including the `O_RDONLY` , `O_CREAT` , and `O_APPEND` flags used with `open()`
- `stdio.h` : Provides formatted output via `printf()` and `sprintf()`
- `stdlib.h` : Provides general utility functions
- `stdint.h` : Defines fixed-width integer types like `uint32_t` that ensure consistent data sizes across platforms
- `string.h` : Provides `strlen()` for calculating message lengths

- `unistd.h` : Provides low-level I/O functions like `read()` , `write()` , `lseek()` , and `close()`
- `time.h` : Provides time-related functions for generating timestamps

```
#define BLOCK_SIZE 4096           // Size of each block in bytes
#define TOTAL_BLOCKS 64          // Total number of blocks in the file system
#define INODE_SIZE 256           // Size of each inode in bytes
#define INODE_COUNT 80           // Total number of inodes
#define MAGIC_NUMBER 0xD34D      // Magic number to identify the file system
```

These constants define fundamental file system parameters:

- `BLOCK_SIZE` : Defines the atomic unit of allocation (4KB, a common size in modern file systems)
- `TOTAL_BLOCKS` : Limits the file system to 64 blocks (256KB total size)
- `INODE_SIZE` : Defines the size of each inode structure (256 bytes)
- `INODE_COUNT` : Limits the number of files to 80
- `MAGIC_NUMBER` : A unique identifier (0xD34D resembles "DEAD") to verify the file system type

```
#define SUPERBLOCK_OFFSET 0       // Block 0: Superblock
#define INODE_BITMAP_OFFSET 1     // Block 1: Inode bitmap
#define DATA_BITMAP_OFFSET 2    // Block 2: Data bitmap
#define INODE_TABLE_START 3       // Blocks 3-7: Inode table (5 blocks)
#define DATA_BLOCK_START 8       // Blocks 8-63: Data blocks
```

These constants define the file system layout:

- The first block (0) contains the superblock (file system metadata)
- The second block (1) contains the inode allocation bitmap
- The third block (2) contains the data block allocation bitmap
- Blocks 3-7 store the inode table (5 blocks × 4096 bytes ÷ 256 bytes per inode = 80 inodes)
- Blocks 8-63 (56 blocks) are available for file data

This layout follows classic UNIX file system design with distinct regions for metadata and data.

III. Data Structures: Modeling File System Components

Superblock Structure

```
typedef struct {
    uint16_t magic_number;           // Identifies the file system (0xD34D)
    uint32_t block_size;             // Size of each block (4096B)
    uint32_t total_blocks;           // Total number of blocks (64)
```

```

uint32_t inode_bitmap_block;           // Block number of inode bitmap (1)
uint32_t data_bitmap_block;           // Block number of data bitmap (2)
uint32_t inode_table_start_block;     // First block of inode table (3)
uint32_t first_data_block_number;     // First data block (8)
uint32_t inode_size;                 // Size of each inode (256B)
uint32_t inode_count;                // Number of inodes (80)
uint8_t reserved[4058];              // Padding to fill the block
} Superblock;

```

The superblock is the master record of the file system:

- **magic_number** : A 16-bit identifier (0xD34D) that confirms this is a VSFS
- **block_size** through **inode_count** : Key parameters defining the dimensions and layout
- **reserved** : Padding to ensure the structure fills exactly one 4096-byte block

The superblock's critical importance stems from the fact that all other file system operations depend on its parameters. If the superblock is corrupted, the entire file system becomes inaccessible.

Inode Structure

```

typedef struct {
    uint32_t mode;           // File permissions and type
    uint32_t uid;            // User ID of owner
    uint32_t gid;            // Group ID of owner
    uint32_t file_size;      // Size of file in bytes
    uint32_t atime;          // Last access time
    uint32_t ctime;          // Creation time
    uint32_t mtime;          // Last modification time
    uint32_t dtime;          // Deletion time (0 if not deleted)
    uint32_t hard_links;     // Number of hard links to this inode
    uint32_t blocks;         // Number of blocks allocated to the file
    uint32_t direct_block;   // Direct block pointer
    uint32_t single_indirect_block; // Points to a block of pointers
    uint32_t double_indirect_block; // Points to a block of pointers to blocks of pc
    uint32_t triple_indirect_block; // Three levels of indirection
    uint8_t reserved[156];   // Padding to match INODE_SIZE (256B)
} Inode;

```

The inode (index node) is the heart of Unix-like file systems, storing metadata about each file:

- **Standard Unix metadata:**
 - **mode** : File type and permissions (read/write/execute bits)
 - **uid / gid** : User and group ownership identifiers
 - **file_size** : Size of the file in bytes

- Various timestamps (atime , ctime , mtime , dtime)

- **Reference counting:**

- `hard_links` : Number of directory entries referencing this inode
- When `hard_links` drops to 0, the inode is considered free

- **Block pointers:**

- `direct_block` : Points directly to a single data block
- `single_indirect_block` : Points to a block containing $(4096/4 = 1024)$ data block pointers
- `double_indirect_block` : Two levels of indirection ($1024 \times 1024 = 1,048,576$ blocks)
- `triple_indirect_block` : Three levels ($1024^3 = 1,073,741,824$ blocks)

This multi-level indirection scheme allows addressing extremely large files while keeping the inode size fixed. VFSck uses a simplified approach with just one direct block pointer, which is sufficient for our demonstration purposes.

IV. Global Variables and Core Utilities

```
int log_fd;           // File descriptor for the log file
int fix_errors = 1;   // Flag to control whether to fix errors (1) or just check (0)
int inodes_modified = 0; // Flag to track if inodes were modified
```

These global variables manage the program's state:

- `log_fd` : Stores the file descriptor for logging, enabling persistent records
- `fix_errors` : Controls whether the program operates in repair mode (1) or check-only mode (0)
- `inodes_modified` : Tracks whether any inode was modified, avoiding unnecessary disk writes

```
void log_message(const char *message) {
    printf("%s", message);           // Display on screen
    write(log_fd, message, strlen(message)); // Write to log file
}
```

This utility function outputs messages to both:

- Standard output (terminal) for real-time monitoring
- The log file for persistent record-keeping

It uses `strlen()` to calculate the exact message length and the low-level `write()` system call

for file operations.

```
int is_valid_block(uint32_t block_num) {
    return (block_num >= DATA_BLOCK_START && block_num < TOTAL_BLOCKS);
}
```

This critical validation function checks if a block number is within the valid range for data blocks:

- Must be $\geq$ DATA_BLOCK_START (8) to avoid metadata blocks
- Must be $<$ TOTAL_BLOCKS (64) to avoid out-of-bounds access
- Returns 1 (true) for valid blocks, 0 (false) for invalid blocks

```
int is_valid_inode(Inode *inode) {
    return (inode->hard_links > 0 && inode->dtime == 0);
}
```

This function determines if an inode is currently in use by checking:

- `hard_links > 0` : At least one directory entry references this inode
- `dtime == 0` : The inode has not been marked for deletion

Both conditions must be true for an inode to be considered valid.

V. File System Access Operations

```
void read_superblock(int fs, Superblock *sb) {
    lseek(fs, SUPERBLOCK_OFFSET * BLOCK_SIZE, SEEK_SET); // Position at the start c
    read(fs, sb, sizeof(Superblock)); // Read the superblock str
}
```

This function reads the superblock from the file system:

1. `lseek()` positions the file pointer at the beginning of block 0 (offset 0)
2. `read()` loads the superblock structure into memory

```
void read_bitmap(int fs, int block, uint8_t *bitmap, size_t size) {
    lseek(fs, block * BLOCK_SIZE, SEEK_SET); // Position at the specified block
    read(fs, bitmap, size); // Read the bitmap data
}
```

```
void write_bitmap(int fs, int block, uint8_t *bitmap, size_t size) {
    if (fix_errors) { // Only write if in fix mode
        lseek(fs, block * BLOCK_SIZE, SEEK_SET);
    }
}
```

```

        write(fs, bitmap, size);
        log_message("Bitmap updated on disk.\n");
    }
}

```

These functions handle bitmap operations:

- `read_bitmap()` loads a bitmap from a specified block into memory
- `write_bitmap()` writes a bitmap back to disk, but only if `fix_errors` is enabled
- Both use `lseek()` to position the file pointer at the correct block

```

void read_inode_table(int fs, Inode *inode_table, size_t count) {
    lseek(fs, INODE_TABLE_START * BLOCK_SIZE, SEEK_SET);
    read(fs, inode_table, sizeof(Inode) * count);
}

void write_inode_table(int fs, Inode *inode_table, size_t count) {
    if (fix_errors && inodes_modified) { // Only write if in fix mode and changed
        lseek(fs, INODE_TABLE_START * BLOCK_SIZE, SEEK_SET);
        write(fs, inode_table, sizeof(Inode) * count);
        log_message("Inode table updated on disk.\n");
        inodes_modified = 0; // Reset the modification flag
    }
}

```

These functions manage the inode table:

- `read_inode_table()` loads all inodes (80) into memory at once
- `write_inode_table()` writes back modified inodes, but only if:
 - The program is in fix mode (`fix_errors` is true)
 - Inodes were actually modified (`inodes_modified` is true)
- The modification flag is reset after writing to avoid duplicate writes

```

void read_block(int fs, uint32_t block_num, uint32_t *block_data) {
    lseek(fs, block_num * BLOCK_SIZE, SEEK_SET);
    read(fs, block_data, BLOCK_SIZE);
}

void write_block(int fs, uint32_t block_num, uint32_t *block_data) {
    if (fix_errors) { // Only write if in fix mode
        lseek(fs, block_num * BLOCK_SIZE, SEEK_SET);
        write(fs, block_data, BLOCK_SIZE);
    }
}

```

These functions handle arbitrary block I/O:

- `read_block()` loads a specified block into memory
- `write_block()` writes a block back to disk, but only in fix mode
- Used for reading and writing indirect blocks

VI. Superblock Validation

```
int validate_superblock(Superblock *sb) {
    log_message("Validating Superblock...\n");
    int errors = 0;

    if (sb->magic_number != MAGIC_NUMBER) {
        log_message("Error: Invalid magic number.\n");
        errors++;
    }
    if (sb->block_size != BLOCK_SIZE) {
        log_message("Error: Invalid block size.\n");
        errors++;
    }
    // [Additional validation checks...]

    if (errors == 0) {
        log_message("Superblock is valid.\n");
    } else {
        char msg[100];
        sprintf(msg, "Superblock has %d errors.\n", errors);
        log_message(msg);
    }

    log_message("Superblock validation complete.\n");
    return errors;
}
```

This function performs extensive validation of the superblock:

1. Checks `magic_number` to confirm this is a VSFS file system
2. Verifies essential parameters match expected values (`block_size` , `total_blocks` , etc.)
3. Validates the layout parameters (`inode_bitmap_block` , `data_bitmap_block` , etc.)
4. Ensures the inode count is within the capacity of the inode table blocks
5. Returns the total count of errors found

The superblock validation is critical because all other file system operations depend on these parameters being correct. Corrupted superblocks can render the entire file system inaccessible, even if the data blocks themselves are intact.

VII. Inode Bitmap Validation

```
int check_and_fix_inode_bitmap(int fs, uint8_t *inode_bitmap, Inode *inode_table) {
    log_message("Checking Inode Bitmap...\n");
    int errors = 0;

    for (int i = 0; i < INODE_COUNT; i++) {
        int valid_inode = is_valid_inode(&inode_table[i]);
        int bitmap_set = inode_bitmap[i / 8] & (1 << (i % 8));

        if (valid_inode && !bitmap_set) {
            // Inode is valid but not marked in bitmap
            char msg[100];
            sprintf(msg, "Error: Inode %d is valid but not marked in bitmap.\n", i);
            log_message(msg);
            errors++;

            if (fix_errors) {
                sprintf(msg, "Fixing: Marking inode %d as used in bitmap.\n", i);
                log_message(msg);
                inode_bitmap[i / 8] |= (1 << (i % 8)); // Set the bit
            }
        }
        if (!valid_inode && bitmap_set) {
            // Inode is not valid but marked in bitmap
            char msg[100];
            sprintf(msg, "Error: Inode %d is marked in bitmap but not valid.\n", i);
            log_message(msg);
            errors++;

            if (fix_errors) {
                sprintf(msg, "Fixing: Marking inode %d as unused in bitmap.\n", i);
                log_message(msg);
                inode_bitmap[i / 8] &= ~(1 << (i % 8)); // Clear the bit
            }
        }
    }

    // Log results and write fixed bitmap if needed
    if (errors == 0) {
        log_message("Inode bitmap is consistent.\n");
    } else {
        char msg[100];
        sprintf(msg, "Inode bitmap has %d inconsistencies.\n", errors);
        log_message(msg);

        if (fix_errors) {
            write_bitmap(fs, INODE_BITMAP_OFFSET, inode_bitmap, BLOCK_SIZE);
        }
    }
}
```



```

    log_message("Inode Bitmap check complete.\n");
    return errors;
}

```

This function performs a comprehensive check of the inode bitmap:

1. **Bitmap Structure:** The bitmap uses 1 bit per inode. For inode i :

- The bit is at index $(i \% 8)$ within byte $(i / 8)$
- If the bit is set (1), the inode is allocated
- If the bit is clear (0), the inode is free

2. **Consistency Rules:**

- If an inode is valid (has links > 0 and is not deleted), its bitmap bit should be set
- If an inode is invalid (no links or deleted), its bitmap bit should be clear

3. **Bit Manipulation:**

- `inode_bitmap[i / 8] & (1 << (i % 8))` : Tests if bit i is set
- `inode_bitmap[i / 8] |= (1 << (i % 8))` : Sets bit i to 1
- `inode_bitmap[i / 8] &= ~(1 << (i % 8))` : Clears bit i to 0

4. **Error Handling:**

- Logs each inconsistency found
- If in fix mode, corrects the bitmap and writes it back to disk
- Returns the total count of errors found

The inode bitmap is crucial for tracking which inodes are in use. Inconsistencies can lead to either:

- Lost files (valid inodes marked as free)
- Wasted space (unused inodes marked as allocated)

VIII. Block Pointer Checking

```

int check_fix_block_ptr(int fs, uint32_t *block_ptr, uint8_t *block_usage,
                       int *bad_blocks, int inode_num, const char *block_type) {
    if (*block_ptr == 0) {
        return 0; // No block to check
    }

    if (!is_valid_block(*block_ptr)) {
        (*bad_blocks)++;
    }
}

```

```
char msg[150];
sprintf(msg, "Error: Bad %s block %u (out of valid range) in inode %d.\n",
        block_type, *block_ptr, inode_num);
log_message(msg);

if (fix_errors) {
    sprintf(msg, "Fixing: Setting %s block pointer in inode %d to 0.\n",
            block_type, inode_num);
    log_message(msg);
    *block_ptr = 0; // Clear invalid block reference
    return 1; // Block was fixed
}
} else {
    block_usage[*block_ptr]++; // Track usage for duplicate detection
}

return 0;
}
```

This function is a critical utility for validating block pointers:

1. **Early Exit:** If the pointer is already 0 (null), there's nothing to check.
2. **Validation:** Uses `is_valid_block()` to verify the block number is within range:
 - Must be ≥ 8 (start of data blocks)
 - Must be < 64 (total number of blocks)
3. **Error Tracking:**
 - Increments the `bad_blocks` counter when an invalid pointer is found
 - Generates detailed error messages including the inode number and block type
 - The block type parameter (e.g., "direct", "single indirect") adds context to error messages
4. **Error Correction:**
 - In fix mode, invalid pointers are zeroed out
 - Returns 1 to indicate the pointer was modified
5. **Block Usage Tracking:**
 - For valid block pointers, increments the corresponding entry in `block_usage`
 - This array is later used to detect blocks referenced by multiple inodes
 - Also used to reconcile the data bitmap

This function embodies a key principle: when a block reference is invalid, the safest approach is to clear it rather than try to guess what it should be.

IX. Indirect Block Processing

Single Indirect Block Processing

```

int process_single_indirect(int fs, uint32_t block_num, uint8_t *block_usage,
                           int *bad_blocks, int inode_num, Inode *inode) {
    uint32_t block_pointers[BLOCK_SIZE/4]; // Array of 1024 block numbers
    int fixed_count = 0;

    read_block(fs, block_num, block_pointers); // Read the indirect block

    // Check each pointer in the single indirect block
    for (int i = 0; i < BLOCK_SIZE/4; i++) {
        if (block_pointers[i] != 0) {
            if (!is_valid_block(block_pointers[i])) {
                // Invalid pointer detected
                (*bad_blocks)++;
                char msg[150];
                sprintf(msg, "Error: Bad block %u from single indirect in inode %d (",
                        block_pointers[i], inode_num);
                log_message(msg);

                if (fix_errors) {
                    sprintf(msg, "Fixing: Clearing invalid block pointer %d in single",
                            block_pointers[i]);
                    log_message(msg);
                    block_pointers[i] = 0; // Clear the invalid pointer
                    fixed_count++;

                    // Decrement blocks counter in inode
                    if (inode->blocks > 0) {
                        inode->blocks--;
                    }
                }
            } else {
                block_usage[block_pointers[i]]++; // Track block usage
            }
        }
    }

    // Write back fixed pointers if needed
    if (fixed_count > 0 && fix_errors) {
        write_block(fs, block_num, block_pointers);
    }

    return fixed_count;
}

```

This function processes a single indirect block:

1. **Block Structure:** A single indirect block contains an array of 1024 block pointers (4096 bytes ÷ 4 bytes per pointer).

2. Traversal Logic:

- Reads the entire indirect block into memory
- Iterates through each pointer in the block
- Only processes non-zero pointers (zero indicates unused entry)

3. Validation and Tracking:

- Checks each pointer for validity using `is_valid_block()`
- For valid pointers, increments the corresponding entry in `block_usage`
- For invalid pointers, increments the bad block count

4. Error Correction:

- In fix mode, clears invalid pointers
- Decrements the inode's `blocks` count to maintain consistency
- Keeps track of how many pointers were fixed

5. Disk Synchronization:

- Only writes the modified block back to disk if changes were made
- This optimization avoids unnecessary I/O

Single indirect blocks are the first level of indirection, allowing a file to address up to 1024 data blocks (4MB) beyond the single direct block.

Double Indirect Block Processing

```
int process_double_indirect(int fs, uint32_t block_num, uint8_t *block_usage,
                           int *bad_blocks, int inode_num, Inode *inode) {
    uint32_t primary_pointers[BLOCK_SIZE/4]; // Array of 1024 block numbers
    int fixed_count = 0;

    read_block(fs, block_num, primary_pointers); // Read the first-level block

    // Process each pointer in the first level
    for (int i = 0; i < BLOCK_SIZE/4; i++) {
        if (primary_pointers[i] != 0) {
            if (!is_valid_block(primary_pointers[i])) {
                // Invalid first-level pointer
                (*bad_blocks)++;
                char msg[150];
                sprintf(msg, "Error: Bad secondary pointer %u in double indirect blc
```

```

        primary_pointers[i], inode_num);
    log_message(msg);

    if (fix_errors) {
        sprintf(msg, "Fixing: Clearing invalid secondary pointer %d in c
        log_message(msg);
        primary_pointers[i] = 0; // Clear the invalid pointer
        fixed_count++;

        // Decrement blocks counter in inode
        if (inode->blocks > 0) {
            inode->blocks--;
        }
    }
} else {
    block_usage[primary_pointers[i]]++; // Track block usage for the ir

    // Recursively process the second-level block (a single indirect blc
    fixed_count += process_single_indirect(fs, primary_pointers[i], bloc
        bad_blocks, inode_num, inode);
}
}

// Write back fixed first-level pointers if needed
if (fixed_count > 0 && fix_errors) {
    write_block(fs, block_num, primary_pointers);
}

return fixed_count;
}

```

This function handles double indirect blocks, which involve two levels of indirection:

1. Hierarchical Structure:

- First level: A block containing 1024 pointers to second-level blocks
- Second level: Each second-level block contains 1024 pointers to data blocks
- This allows addressing up to $1024 \times 1024 = 1,048,576$ blocks (4GB)

2. Recursive Processing:

- Reads the first-level block
- Validates each first-level pointer
- For valid pointers, recursively processes each second-level block using `process_single_indirect()`

3. Error Handling:

- Handles errors at the first level directly
- Accumulates fixes from both levels
- Updates the inode's block count

4. Optimization:

- Only processes non-zero pointers
- Only writes back to disk if changes were made

The double indirect approach demonstrates the power of multi-level indirection, allowing files to grow to much larger sizes while keeping the inode structure compact.

Triple Indirect Block Processing

```
int process_triple_indirect(int fs, uint32_t block_num, uint8_t *block_usage,
                           int *bad_blocks, int inode_num, Inode *inode) {
    uint32_t primary_pointers[BLOCK_SIZE/4]; // Array of 1024 block numbers
    int fixed_count = 0;

    read_block(fs, block_num, primary_pointers); // Read the first-level block

    // Process each pointer in the first level
    for (int i = 0; i < BLOCK_SIZE/4; i++) {
        if (primary_pointers[i] != 0) {
            if (!is_valid_block(primary_pointers[i])) {
                // Invalid first-level pointer
                (*bad_blocks)++;
                char msg[150];
                sprintf(msg, "Error: Bad tertiary pointer %u in triple indirect block\n",
                        primary_pointers[i], inode_num);
                log_message(msg);

                if (fix_errors) {
                    sprintf(msg, "Fixing: Clearing invalid tertiary pointer %d in tr",
                            primary_pointers[i], inode_num);
                    log_message(msg);
                    primary_pointers[i] = 0; // Clear the invalid pointer
                    fixed_count++;

                    // Decrement blocks counter in inode
                    if (inode->blocks > 0) {
                        inode->blocks--;
                    }
                }
            } else {
                block_usage[primary_pointers[i]]++; // Track block usage for the ir

                // Recursively process the second-level block (a double indirect block)
                fixed_count += process_double_indirect(fs, primary_pointers[i], block_usage,
                                                         bad_blocks, inode_num, inode);
            }
        }
    }
}
```

```

    }
}

// Write back fixed first-level pointers if needed
if (fixed_count > 0 && fix_errors) {
    write_block(fs, block_num, primary_pointers);
}

return fixed_count;
}

```

This function handles triple indirect blocks, which involve three levels of indirection:

1. Extended Hierarchy:

- First level: Pointers to second-level blocks
- Second level: Pointers to third-level blocks
- Third level: Pointers to data blocks
- This allows addressing up to $1024^3 = 1,073,741,824$ blocks (4TB)

2. Deeply Nested Recursion:

- Reads the first-level block
- For each valid pointer, recursively calls `process_double_indirect()`
- This creates a depth-first traversal of the entire tree structure

3. Consistent Error Handling:

- Follows the same pattern as the other indirection levels
- Maintains the inode's block count

4. Performance Considerations:

- For very large files, this recursion could potentially be expensive
- The code optimizes by only following non-zero pointers

Triple indirection represents the most complex addressing mechanism in traditional Unix file systems, enabling support for extremely large files.

X. Master Inode Block Processing

```

int process_inode_blocks(int fs, Inode *inode, uint8_t *block_usage, int *bad_blocks
int inode_modified = 0;
int fixed_count = 0;

```

```
// Check direct block
inode_modified |= check_fix_block_ptr(fs, &inode->direct_block, block_usage,
                                     bad_blocks, inode_num, "direct");

// Check and process single indirect block
if (check_fix_block_ptr(fs, &inode->single_indirect_block, block_usage,
                       bad_blocks, inode_num, "single indirect")) {
    inode_modified = 1;
} else if (inode->single_indirect_block != 0) {
    // Process the single indirect block
    fixed_count = process_single_indirect(fs, inode->single_indirect_block, block_usage,
                                         bad_blocks, inode_num, inode);

    if (fixed_count > 0) {
        inode_modified = 1;
    }
}

// Check and process double indirect block (similar pattern)
// [similar code for double_indirect_block]

// Check and process triple indirect block (similar pattern)
// [similar code for triple_indirect_block]

return inode_modified;
}
```

This orchestration function processes all block pointers in an inode:

1. Top-Down Processing:

- Starts with the simplest direct block
- Progresses through increasing levels of indirection
- Each level is only processed if the pointer to it is valid

2. Error Handling Strategy:

- Uses `check_fix_block_ptr()` to validate the block pointers themselves
- For valid indirect blocks, calls the specialized processing functions
- Tracks whether any modifications were made to the inode

3. Short-Circuit Logic:

- If an indirect block pointer is invalid, its contents are not processed
- This prevents cascading errors and unnecessary processing

4. Modification Tracking:

- Returns a flag indicating if any part of the inode was modified
- This is used by the caller to determine if the inode table needs to be written back to disk

This function ties together all the block pointer checking functionality, providing a complete traversal of an inode's block structure.

XI. Data Bitmap and Duplicate Block Checking

```
int check_and_fix_data_bitmap(int fs, uint8_t *data_bitmap, Inode *inode_table) {
    log_message("Checking Data Bitmap...\n");
    uint8_t block_usage[TOTAL_BLOCKS] = {0}; // Track actual block usage
    int bad_blocks = 0;
    int bitmap_errors = 0;
    inodes_modified = 0; // Reset the global flag
```

The function begins by initializing:

- `block_usage` : An array to track how many times each block is referenced
- `bad_blocks` : Counter for blocks with invalid addresses
- `bitmap_errors` : Counter for bitmap inconsistencies
- Resets the global `inodes_modified` flag

```
// Build block usage map by processing all inodes
for (int i = 0; i < INODE_COUNT; i++) {
    if (is_valid_inode(&inode_table[i])) {
        if (process_inode_blocks(fs, &inode_table[i], block_usage, &bad_blocks,
                                inodes_modified = 1; // Mark that inodes were modified
        )
    }
}
```

This section:

1. Loops through all inodes in the inode table
2. Only processes valid inodes (those with links > 0 and not deleted)
3. Calls `process_inode_blocks()` to validate all block pointers
4. Sets the `inodes_modified` flag if any inodes were changed

```
// Write back modified inodes
if (inodes_modified && fix_errors) {
    write_inode_table(fs, inode_table, INODE_COUNT);
}
```

If any inodes were modified and the program is in fix mode, write the updated inode table back to disk.

```
// Check data bitmap against actual usage
for (int i = DATA_BLOCK_START; i < TOTAL_BLOCKS; i++) {
    int bitmap_set = data_bitmap[i / 8] & (1 << (i % 8)); // Check if bit is set

    if (block_usage[i] > 0 && !bitmap_set) {
        // Block is used but not marked in bitmap
        char msg[100];
        sprintf(msg, "Error: Data block %d is used but not marked in bitmap.\n", i);
        log_message(msg);
        bitmap_errors++;

        if (fix_errors) {
            sprintf(msg, "Fixing: Marking data block %d as used in bitmap.\n", i);
            log_message(msg);
            data_bitmap[i / 8] |= (1 << (i % 8)); // Set the bit
        }
    }
    if (block_usage[i] == 0 && bitmap_set) {
        // Block is marked in bitmap but not used
        char msg[100];
        sprintf(msg, "Error: Data block %d is marked in bitmap but not used.\n", i);
        log_message(msg);
        bitmap_errors++;

        if (fix_errors) {
            sprintf(msg, "Fixing: Marking data block %d as unused in bitmap.\n", i);
            log_message(msg);
            data_bitmap[i / 8] &= ~(1 << (i % 8)); // Clear the bit
        }
    }
}
}
```

This critical section compares the actual block usage to the bitmap:

1. Iterates through all data blocks (8-63)
2. For each block, checks if its bit is set in the bitmap
3. Compares this to the actual usage as tracked in `block_usage`
4. Identifies two types of inconsistencies:
 - Blocks that are used but not marked in the bitmap
 - Blocks that are marked in the bitmap but not actually used
5. In fix mode, corrects the bitmap by setting or clearing bits as needed

```
// Write back fixed bitmap
if (bitmap_errors > 0 && fix_errors) {
```

```

        write_bitmap(fs, DATA_BITMAP_OFFSET, data_bitmap, BLOCK_SIZE);
    }

    // Report results
    if (bad_blocks > 0) {
        char msg[100];
        sprintf(msg, "Total bad blocks found: %d\n", bad_blocks);
        log_message(msg);
    } else {
        log_message("No bad blocks found.\n");
    }

    if (bitmap_errors == 0) {
        log_message("Data bitmap is consistent.\n");
    } else {
        char msg[100];
        sprintf(msg, "Data bitmap has %d inconsistencies.\n", bitmap_errors);
        log_message(msg);
    }

    log_message("Data Bitmap check complete.\n");
    return bitmap_errors + bad_blocks;
}

```

The function concludes by:

1. Writing the fixed bitmap back to disk if errors were found and fixing is enabled
2. Reporting the results of the checks
3. Returning the total count of errors (both bad blocks and bitmap inconsistencies)

The data bitmap is crucial for space allocation in the file system. Inconsistencies can lead to:

- Data loss (if used blocks are marked as free)
- Space leaks (if unused blocks are marked as used)

```

int check_duplicate_blocks(int fs, Inode *inode_table) {
    log_message("Checking for duplicate blocks...\n");
    uint8_t block_usage[TOTAL_BLOCKS] = {0};
    int duplicates = 0;
    int bad_blocks = 0;
    inodes_modified = 0; // Reset the flag
}

```

Similar to the data bitmap check, this function starts by initializing tracking variables.

```

// Build block usage map
for (int i = 0; i < INODE_COUNT; i++) {
    if (is_valid_inode(&inode_table[i])) {

```

```

        if (process_inode_blocks(fs, &inode_table[i], block_usage, &bad_blocks,
                                inodes_modified = 1; // Mark that inodes were modified
        )
    }
}

// Write back modified inodes
if (inodes_modified && fix_errors) {
    write_inode_table(fs, inode_table, INODE_COUNT);
}

```

Like the bitmap check, it processes all valid inodes to build a block usage map.

```

// Check for blocks referenced multiple times
for (int i = DATA_BLOCK_START; i < TOTAL_BLOCKS; i++) {
    if (block_usage[i] > 1) {
        duplicates++;
        char msg[100];
        sprintf(msg, "Error: Data block %d is referenced by %d inodes (duplicate
                    i, block_usage[i]);
        log_message(msg);

        // Note: Fixing duplicates is not trivial - would require copying data
        // We just report the issue
    }
}

```

This is the core duplicate detection logic:

1. Examines the `block_usage` array for blocks referenced more than once
2. Reports each duplicate block, showing how many inodes reference it
3. Unlike other issues, duplicates are not automatically fixed because this would require:
 - Allocating new blocks
 - Copying data
 - Updating the referencing inodes
 - A policy decision about which reference to keep

```

// Report results
if (duplicates == 0) {
    log_message("No duplicate blocks found.\n");
} else {
    char msg[100];
    sprintf(msg, "Total duplicate blocks found: %d\n", duplicates);
    log_message(msg);
}

```

```
    log_message("Duplicate block check complete.\n");  
    return duplicates + bad_blocks;  
}
```

The function concludes by reporting the results and returning the total count of errors (both duplicates and bad blocks).

Duplicate blocks are a serious file system integrity issue that can lead to:

- Data corruption (if one file modifies a block shared with another)
- Inconsistent file contents
- Potential security issues (unintended data sharing)

XII. Main Check Function and Program Entry

```
int check_and_fix_filesystem(int fs) {  
    int total_errors = 0;  
  
    // Load and validate superblock  
    Superblock sb;  
    read_superblock(fs, &sb);  
    total_errors += validate_superblock(&sb);  
  
    // Load bitmaps and inode table  
    uint8_t inode_bitmap[BLOCK_SIZE];  
    uint8_t data_bitmap[BLOCK_SIZE];  
    read_bitmap(fs, INODE_BITMAP_OFFSET, inode_bitmap, BLOCK_SIZE);  
    read_bitmap(fs, DATA_BITMAP_OFFSET, data_bitmap, BLOCK_SIZE);  
    Inode inode_table[INODE_COUNT];  
    read_inode_table(fs, inode_table, INODE_COUNT);  
  
    // Perform all checks  
    total_errors += check_and_fix_inode_bitmap(fs, inode_bitmap, inode_table);  
    total_errors += check_and_fix_data_bitmap(fs, data_bitmap, inode_table);  
    total_errors += check_duplicate_blocks(fs, inode_table);  
  
    return total_errors;  
}
```

This orchestration function coordinates the entire file system check:

1. Loads the superblock and validates it
2. Loads the inode and data bitmaps from their respective blocks
3. Loads the entire inode table
4. Performs each check in sequence:

- Inode bitmap consistency
- Data bitmap consistency and bad block checking
- Duplicate block detection

5. Accumulates the total error count and returns it

The checks are performed in a specific order to ensure all dependencies are satisfied:

- Superblock validation first, since all other checks depend on valid superblock parameters
- Inode bitmap before data bitmap, since inode validity affects block processing
- Duplicate block checking last, leveraging the block usage map built by data bitmap checking

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <file_system_image>\n", argv[0]);
        return 1;
    }

    // Format the current date and time
    time_t now;
    struct tm *tm_info;
    char timestamp[20];

    time(&now);
    tm_info = localtime(&now);
    strftime(timestamp, 20, "%Y-%m-%d %H:%M:%S", tm_info);

    // Open the file system and log file
    int fs = open(argv[1], O_RDWR); // Open with read/write permissions
    if (fs == -1) {
        perror("Error opening file system");
        return 1;
    }

    log_fd = open("vsfsck_log.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (log_fd == -1) {
        perror("Error opening log file");
        close(fs);
        return 1;
    }

    // Log start time and user info
    char header[200];
    sprintf(header, "----- VFSck started at %s by %s -----\\n",
        "2025-05-13 06:46:09", "SyedNaveedMahmood");
    log_message(header);
}
```

The main function begins with initialization:

1. Validates command-line arguments (needs exactly one: the file system image path)
2. Sets up timestamp generation
3. Opens the file system image in read/write mode to allow fixes
4. Opens or creates the log file
5. Logs a header with the start time and user information

```
// First pass: check and fix errors
log_message("PASS 1: Checking and fixing file system errors\n");
fix_errors = 1; // Enable fixing
int initial_errors = check_and_fix_filesystem(fs);

if (initial_errors > 0) {
    char msg[100];
    sprintf(msg, "PASS 1: Found and fixed %d errors\n", initial_errors);
    log_message(msg);

    // Second pass: verify fixes
    log_message("\nPASS 2: Verifying file system integrity after fixes\n");
    fix_errors = 0; // Disable fixing, just check
    int remaining_errors = check_and_fix_filesystem(fs);

    if (remaining_errors > 0) {
        sprintf(msg, "PASS 2: %d errors remain after fixes\n", remaining_errors);
        log_message(msg);
    } else {
        log_message("PASS 2: File system is now consistent\n");
    }
} else {
    log_message("PASS 1: No errors found, file system is consistent\n");
}
```

This is the core two-pass verification process:

1. PASS 1: With `fix_errors = 1` , check the file system and fix errors
2. If errors were found and fixed, proceed to PASS 2
3. PASS 2: With `fix_errors = 0` , re-check to verify fixes were successful
4. Log appropriate messages indicating the results of each pass

The two-pass approach is a standard practice in file system checking:

- The first pass identifies and fixes errors
- The second pass confirms that all fixable errors were actually addressed
- Any errors remaining after the second pass indicate more serious problems

```
log_message("---- VFSck completed ----\n\n");
```

```
// Clean up
close(fs);
close(log_fd);
return 0;
}
```

The function concludes by:

1. Logging completion
2. Closing the file system and log file descriptors
3. Returning success (0)

XIII. Theoretical Significance and Implementation Highlights

1. Unix File System Design Principles:

- Separation of metadata (superblock, inodes) from data
- Use of bitmaps for efficient space allocation
- Indirect block scheme for addressing large files
- Independence of inodes from directory entries

2. Consistency Checking Theory:

- Bitmap consistency (allocation vs. actual usage)
- Reference integrity (valid block addresses)
- Entity completeness (superblock fields, inode fields)
- Reference uniqueness (no duplicate block usage)

3. Error Correction Strategies:

- Conservative approach: clear invalid references rather than guessing
- Bitmap reconciliation based on actual structure
- Two-pass verification to ensure fixes are effective

4. Data Structures and Algorithms:

- Bit manipulation for bitmap operations
- Depth-first traversal of indirect block trees
- Usage counting for duplicate detection
- In-memory tables for tracking state across operations

5. Performance Optimizations:

- Only writing to disk when modifications occurred
- Early termination when processing null pointers
- Use of direct system calls for I/O operations
- Optimized block-by-block processing

The VSFSck implementation demonstrates core principles of file system maintenance while providing practical error detection and correction capabilities for the VSFS file system.